

# Logic Programming Languages: Lists in Prolog

BBM 301 – Programming Languages

# Lists Prolog Exercises

Credit to : Blackburn, Patrick, Johan Bos and Kristina Striegnitz. *Learn Prolog Now*. London: College Publications, 2006. (ISBN 9781904987178) Also available at <http://www.learnprolognow.org>

# Are you a member of a list?

- Either X is the head
- Or x is a member of the tail
- Remember the empty list has no Head and Tail so the empty tail won't fit `member(X,[X|T])`.
- `member(X,[X|T]).`  
    `member(X,[H|T]) :- member(X,T).`
- Because you don't care about H in second or T in first, replace with `_`
- `member(X,[X|_]).`  
    `member(X,[_|T]) :- member(X,T).`

# Recurse down a list

- Change a list of a's to b's
- Goal:
  - Yes to
    - $a2b([a,a,a,a],[b,b,b,b])$ .
    - $a2b([a,a,a],[b,b,b])$ .
    - $a2b([],[])$ .
  - No to
    - $a2b([a,c,a,a],[b,b,5,4])$ .
    - $a2b([a,a,a,a],[b,b,b])$ .

# Recurse a2b

- The empty list is a non-recursive true a2b fact
  - $a2b([],[])$ .
- The head of one list being a and the other being b plus the rest of each list fitting a2b
  - $a2b([a|Ta],[b|Tb]) \text{ :- } a2b(Ta,Tb)$ .
- $a2b([a,a,a,a],X)$ .  
 $X = [b,b,b,b]$ .
- $a2b(X,[b,b,b,b])$ .  
 $X = [a,a,a,a]$

# Translate a list

```
tran(eins,one).  
tran(zwei,two).  
tran(drei,three).  
tran(vier,four).  
tran(fuenf,five).  
tran(sechs,six).  
tran(sieben,seven).  
tran(acht,eight).  
tran(neun,nine).
```

Create `listtran([eins,neun,zwei],X)`. That will return the translation: `X = [one,nine,two]`.

# Translation Answer

- Good translation: A list of no items translates to a list of no items
  - `listtran([],[]).`
- Another good translation: the head of the german list being part of a tran fact with the head of the english list plus the rest of the lists being a translation as well.
- `listtran([Hg|Tg], [He|Te]):- tran(Hg,He), listtran(Tg,Te).`

# Append 2 lists

- Append is true when nothing added to a List yields that List
  - `append([],L,L).`
- Append is true when the first element of the list = the first element of another list, and when the tail of the first list is appended to the tail of the result.
  - `append([H|T],L2,[H|L3]) :- append(T,L2,L3).`

# Trace append

```
append([a, b, c], [1, 2, 3], _G518)
  append([b, c], [1, 2, 3], _G587)
    append([c], [1, 2, 3], _G590)
      append([], [1, 2, 3], _G593)
        append([], [1, 2, 3], [1, 2, 3])
          append([c], [1, 2, 3], [c, 1, 2, 3])
            append([b, c], [1, 2, 3], [b, c, 1, 2, 3])
              append([a, b, c], [1, 2, 3], [a, b, c, 1, 2, 3])
```

# prefix

- How to get all the elements that are prefixes in the list?
- `prefix(X,[a,b,c,d]).`

`X = [] ;`

`X = [a] ;`

`X = [a,b] ;`

`X = [a,b,c] ;`

`X = [a,b,c,d] ;`

# prefix

- `prefix(P,L):- append(P,_,L).`

# Exercise: Query for Suffix

- ?- suffix(X,[a,b,c,d]).

X = [a,b,c,d] ;

X = [b,c,d] ;

X = [c,d] ;

X = [d] ;

X = [] ;

# Reversing a list

- Naïve:

nrev([],[]).

nrev([H|T],R):-

    nrev( T,RevT), append(RevT,[H], R) .

# Use an accumulator

`accRev([],A,A).`

`accRev([H|T],A,R):- accRev(T,[H|A],R).`

`accRev([1,2,3],[],X).`

Call: (6) `accRev([1, 2, 3], [], _G2326) ? creep`

Call: (7) `accRev([2, 3], [1], _G2326) ? creep`

Call: (8) `accRev([3], [2, 1], _G2326) ? creep`

Call: (9) `accRev([], [3, 2, 1], _G2326) ? creep`

Exit: (9) `accRev([], [3, 2, 1], [3, 2, 1]) ? creep`

Exit: (8) `accRev([3], [2, 1], [3, 2, 1]) ? creep`

Exit: (7) `accRev([2, 3], [1], [3, 2, 1]) ? creep`

Exit: (6) `accRev([1, 2, 3], [], [3, 2, 1]) ? creep`

`X = [3, 2, 1].`

# How the Accumulator Works

- Seed the accumulator with an empty list
- It grows as it is called recursively
- At the base case, transfer the accumulator over to a variable that will pass back.

# Accumulate a List of Throws

- `throwUntil_11WList(A,[11 | A]):- throw2(11).`
- `throwUntil_11WList(PriorRolls, R ):-  
throw2(Y), Y \= 11,  
throwUntil_11WList([Y | PriorRolls],R).`
- Call as : `throwUntil_11WList([],X).`
- When the throw is 11 (base case), transfer the accumulator to the result.
- When the throw is not 11, insert the throw at the head of the list.

# Accumulating a List

- Accumulated text:
  - descend2(X,Y, parentChild(X,Y)):- child(X,Y).
  - descend2(X,Y, parentChild(X,I,L)):- child(X,I),  
– descend2(I,Y,L).
- Accumulate a list:
  - descendList(X,Y,L,[L | child(X,Y) ]):- child(X,Y).
  - descendList(X,Y,L,A):- child(X,I),  
– descendList(I,Y,[L,child(X,I) ],A).
- Call as:
  - descendList(anne,donna,anne,X).

# Accumulating a List

- Like counting,
  - Need to seed the first item in the list.
  - Need to transfer the accumulation to a result variable.

# Exercise:

## Accumulate the Path you Walk

- Build a list of the edges you travel
- `connectedPathtoList(a,e, a,X).`
  - $X = [[a, \text{edge}(a, b)] \mid \text{edge}(b, e)] ;$
  - $X = [[[a, \text{edge}(a, b)], \text{edge}(b, d)] \mid \text{edge}(d, e)] .$

# Summary

- Syntax for list creation
- [ H | T ] use to split off the first element
  - T is a list
- Determine whether member
- Append
- Reverse using accumulation
- Build a list using accumulation

# Recursion

## Recap

A notion is **recursive** if it is defined in terms of itself.

- We can define recursive data types, recursive functions, or recursive relations.
- In general, a recursive definition consists of
  - 0 or more **base cases**
  - 1 or more **recursive cases**

In Prolog

- we can work with recursive data: trees, lists, etc.
- we can define recursive relations

# Recursive datatypes and predicates

## Lists

**List** = predefined datatype of Prolog. A list is either

- The empty list `[]` (base case)
- `[term | list]` (recursive case)

### Some definitions of predicates and relations for lists

```
% isList(lst) holds if lst is a list
```

```
isList([]). % base case
```

```
isList([_|T]) :- isList(T). % recursive case
```

```
% member(term, lst) holds if term is in list lst
```

```
member(X, [X|_]). % base case: term is the head of the list
```

```
member(X, [_|T]) :- member(X, T). % recursive case
```

# Remarks

- 1 All variables must start with `_` or with an uppercase letter.
- 2 Like in Haskell, we can use the anonymous variable `_` which matches every term.
- 3 Variables can occur many times, both in the head and in the body of a clause. We could have written

```
member (X, [Y | _]) :- X=Y.
```

but Prolog allows to be more concise, and write

```
member (X, [X | _]) .
```

- 4 `member` is predefined in Prolog.

# Remarks

## More about list membership tests

Prolog tries to find if a term is in a list by applying the rules in the order from the program:

- 1 `member (X, [X | T]) .` (base case)
- 2 `member (X, [_ | T]) :- member (X, T) .` (recursive case)

In the recursive case, the list gets shorter and shorter.

- The list can not be shortened indefinitely  $\Rightarrow$  computation will terminate.

Prolog stops computation in 2 situations:

- 1 when it encounters a list for which base case holds  $\Rightarrow$  it returns `true`.
- 2 when it reaches the empty list  $\Rightarrow$  no more rules are applicable  $\Rightarrow$  it returns `false`.

# Quiz

What happens when Prolog is asked to answer the following queries:

```
?- member(X, [a, b, c]) .  
?- member(a, X) .  
?- member(X, Y) .  
?- member(X, _) .  
?- member(_, Y) .  
?- member(_, _) .
```

Note that some queries have multiple answers.

# Quiz

What happens when Prolog is asked to answer the following queries:

?- member (X, [a, b, c]) .

?- member (a, X) .

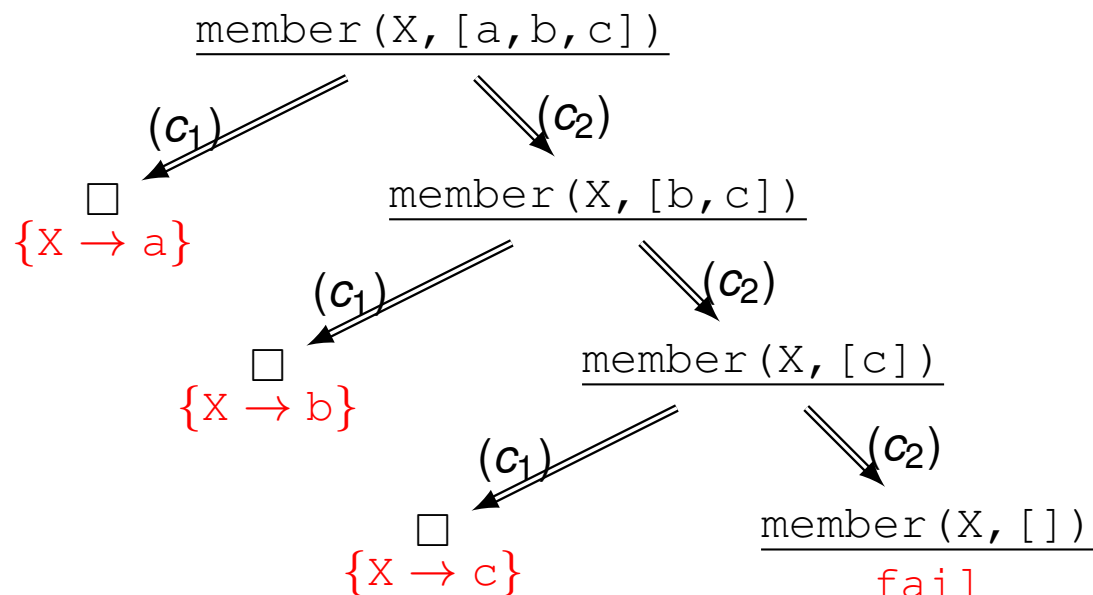
?- member (X, Y) .

?- member (X, \_) .

?- member (\_, Y) .

?- member (\_, \_) .

Note that some queries have multiple answers.



# Recursive datatypes and predicates

## Binary trees

We can work with binary trees defined by the grammar

```
btree ::= null | bt(string,btree,btree)
```

### Some definitions of predicates and relations for binary trees

```
% isBT(bt) holds if bt is a binary tree
```

```
isBT(null).
```

```
isBT(bt(S,T1,T2)) :- string(S), isBT(T1), isBT(T2).
```

```
% toList(bt, lst) holds if lst is the list of strings from bt
```

```
toList(null, []).
```

```
toList(bt(S,T1,T2), Lst) :-  
    toList(T1, L1),  
    toList(T2, L2),  
    append(L1, [S|L2], Lst).
```

# Recursion

## Termination problems

**Termination** = property of a program to stop after a finite number of computational steps.

- Some programs do not terminate

```
parent (X, Y) :- son (Y, X) .
```

```
son (Y, X) :- parent (X, Y) .
```

Reason: these definitions are circular.

⇒ avoid circular definitions!

- The following program does not terminate because it is left-recursive:

```
man (X) :- man (Y) , parent (X, Y) .
```

```
man (adam) .
```

⇒ left-recursion should be used with care!

# Recursion

## Termination problems

Program clauses (=rules and facts) are applied in the order in which they are written in the program.

- Intuitive criterion: facts should appear before rules.

Sometimes, rules ordered in a particular way work well only for a particular kind of queries.

### Example

```
isList([_|T]) :- isList(T) .  
isList([]) .
```

is adequate to answer the queries

```
?-isList([1,2,3]) .  
?-isList([]) .  
?-isList(f(1,2)) .
```

but inadequate for `?-isList(X)`.

# Recursion

## Termination problems

Program clauses (=rules and facts) are applied in the order in which they are written in the program.

- Intuitive criterion: facts should appear before rules.

Sometimes, rules ordered in a particular way work well only for a particular kind of queries.

### Example

```
isList([_|T]) :- isList(T) .  
isList([]) .
```

is adequate to answer the queries

```
?-isList([1,2,3]) .  
?-isList([]) .  
?-isList(f(1,2)) .
```

but inadequate for `?-isList(X)`.

**Question:** What answer do you get to `?-isList(X)` if you change the order of clauses in the program?

# Recursion

## The order of building solutions

Many predefined predicates of Prolog make distinction between

**Input parameters (-):** they should have concrete values when the predicate is called.

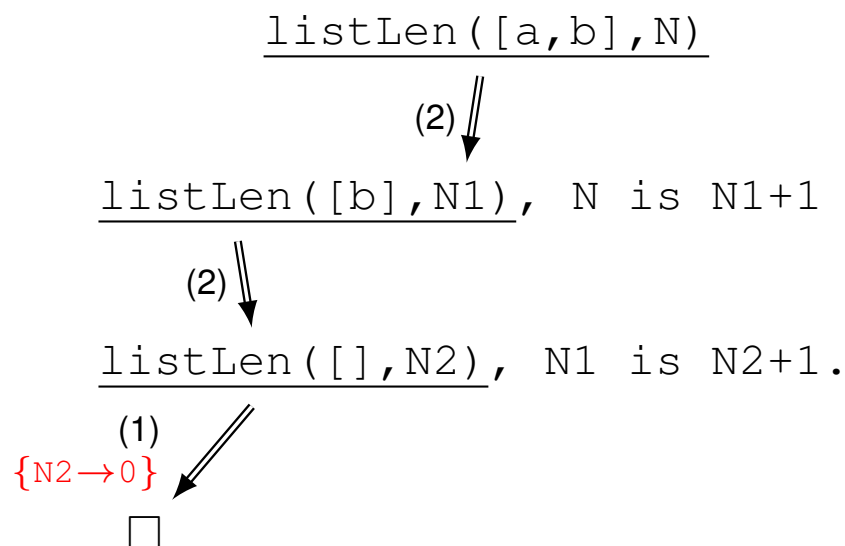
**Output parameters (+):** their values are computed as answers to the query.

**Arbitrary parameters (?):** can be both input and output.

# Recursion

## Example

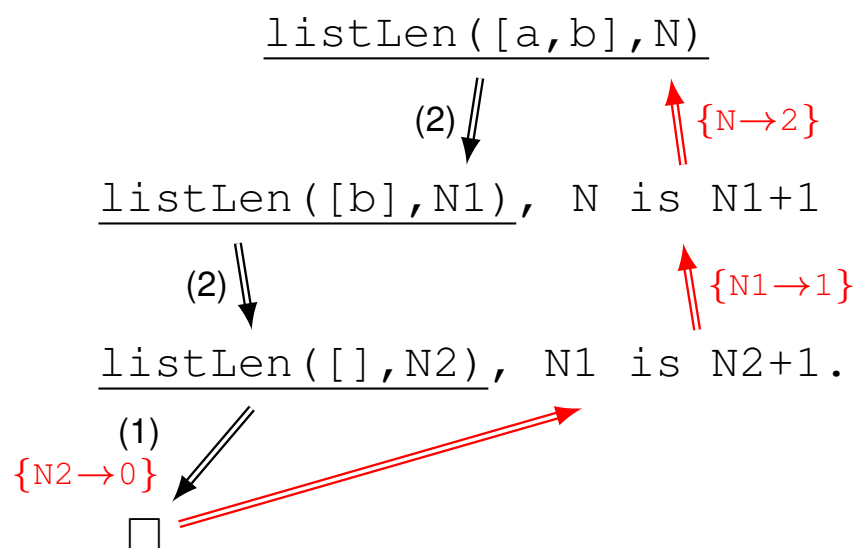
```
% listLen(+List,-N) computes the number N of elements of List.  
% List is an input parameter, and N is output parameter.  
listLen([],0).                                % (1)  
listLen([_|T],N):-listLen(T,N1), N is N1+1.    % (2)
```



# Recursion

## Example

```
% listLen(+List,-N) computes the number N of elements of List.  
% List is an input parameter, and N is output parameter.  
listLen([],0). % (1)  
listLen([_|T],N):-listLen(T,N1), N is N1+1. % (2)
```



Remarks: